Fases del COMPILADOR

Lenguajes Formales y de Programación
Sección A-

Inga. Damaris Campos

Lenguaje Natural – Lenguaje Formal

Desarrollado por el ser humano, como medio de comunicación cotidiana con los demás.

- Se desarrolla por enriquecimiento progresivo, antes de cualquier formalización completa.
- Dificultad o imposibilidad de formalización completa.
- Polisemántica, puede ser ambiguo o interpretarse dependiendo del contexto.
- Poder EXPRESIVO.

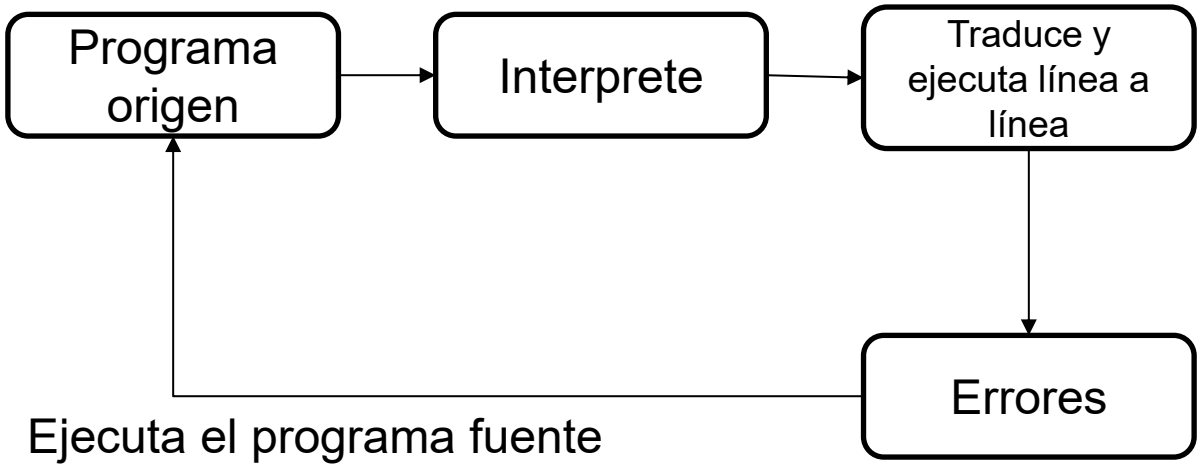
Un lenguaje desarrollado por el hombre, para expresar situaciones específicas en un área de conocimiento.

- Se desarrolla de una teoría preestablecida – ESTRUCTURA.
- Requiere completa formalización, para implementarlo en una construcción computacional.
- La sintaxis debe producir sentencias NO AMBIGUAS.
- Componente semántico MÍNIMO.
- La importancia del rol de los números.
- Poder expresivo LIMITADO!

Fundamentos

- Los compiladores ayudan a promover el uso de lenguajes de alto nivel y con ello se minimiza la sobrecarga de ejecución de los programas escritos en estos lenguajes.
- El conocimiento sobre compiladores es de gran relevancia para un ingeniero ya que amplía su visión al momento de diseñar algoritmos y la codificación de los mismos.

Intérpretes



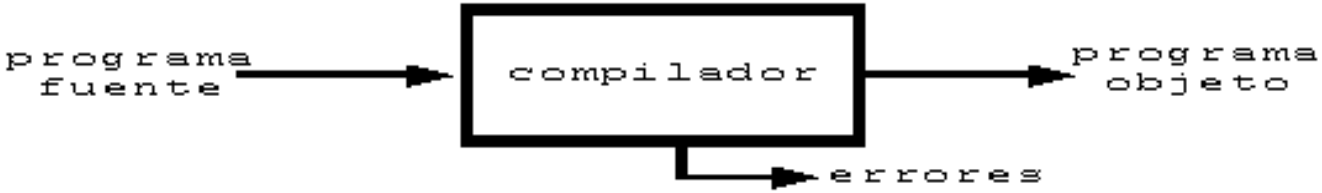
Ejecuta el programa fuente inmediatamente en vez de generar un código objeto que se ejecuta después de que se completa la traducción.

INTERPRETE

- El **intérprete** ejecuta el código según lo va interpretando.
- Cada vez que se escribe una línea el programa comprueba si es correcta, si lo es, la ejecuta.
- La ejecución es interactiva.
- Los intérpretes más puros no guardan copia del programa que se está escribiendo.
- El intérprete siempre debe estar presente.

¿Qué es un compilador (traductor)?

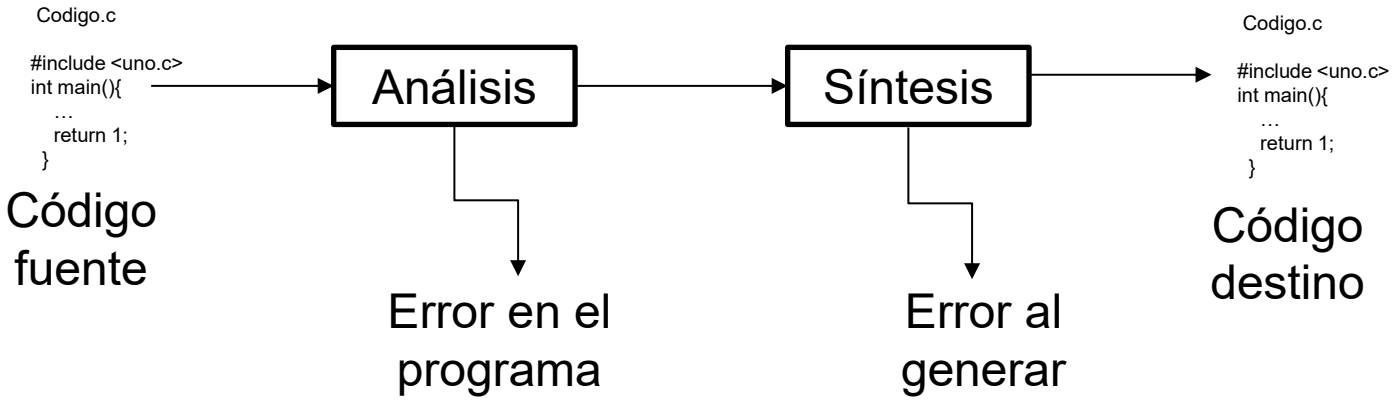
- Programa que lee un programa (fuente) en un lenguaje ASCII y lo traduce a un programa EQUIVALENTE en otro lenguaje (objeto)
- Además:
  - da mensajes de error
  - lleva a cabo determinadas “correcciones” (recuperación de errores)
  - puede optimizar el código generado



Permite programar “independientemente” de la máquina

- Importante, ya que el número de máquinas diferente crece deprisa

Compilación



COMPILADOR

- El **compilador** es el traductor más extendido
- Realiza un análisis y genera un programa ejecutable
- El programa ejecutable, una vez creado, no necesita el compilador para funcionar

COMPILADOR VS INTERPRETE

- **Compilador**

– Se compila una vez, se ejecuta *n* veces

– El proceso de compilación tiene una visión global de todo el programa, por lo cual la gestión de errores es más eficiente.

– La ejecución es más rápida.

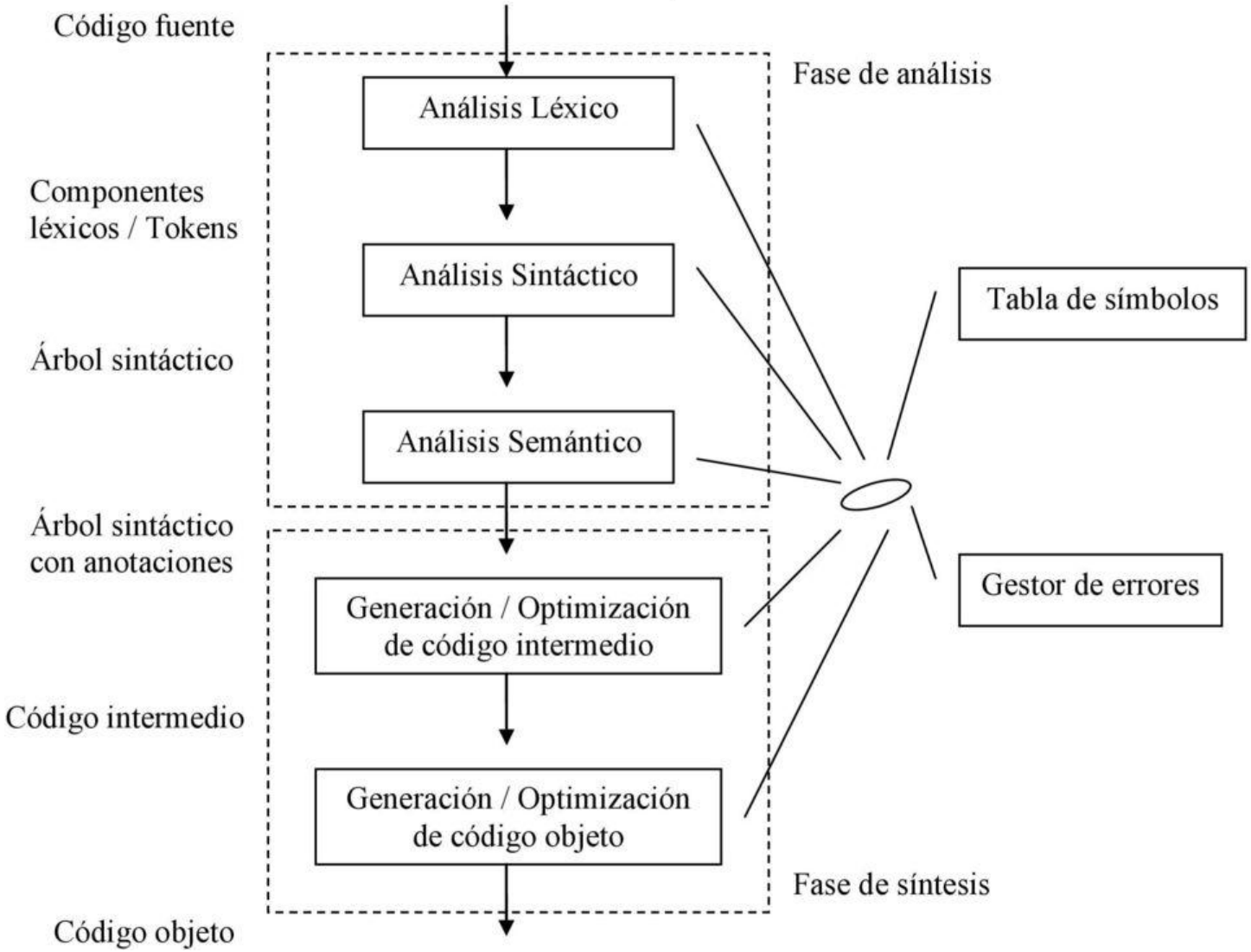
- **Intérprete**

– Se traduce cada vez que se ejecuta

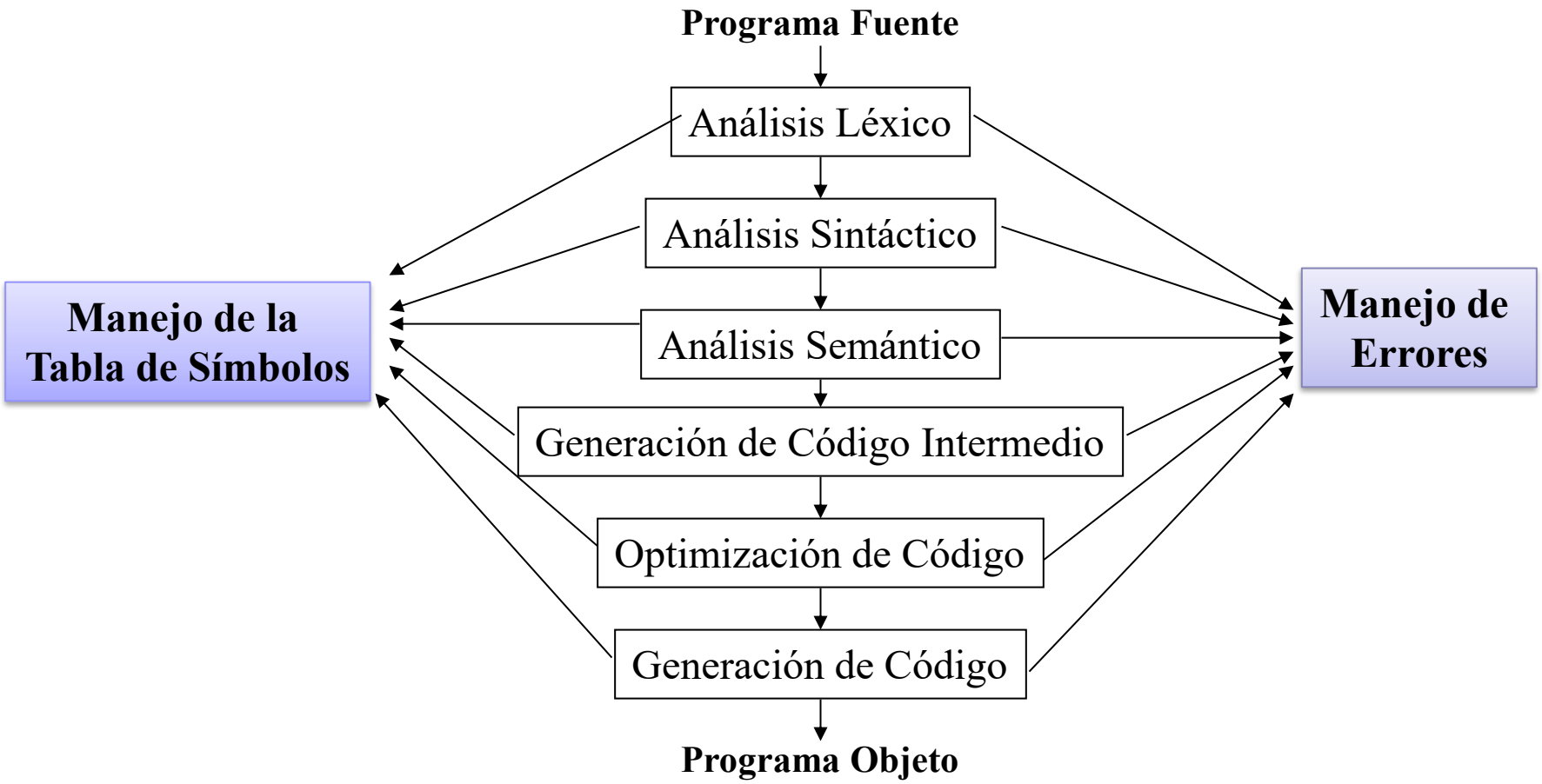
– Permite interaccionar más con el código en tiempo de ejecución.

– Necesita menos memoria

Fases de Compilación



ESTRUCTURA DE UN COMPILADOR



El analizador léxico

- **Lo realiza un “scanner”**
- **El scanner recorre los caracteres de entrada (el fuente) hasta reconocer un “token”**
  - token: unidad léxica indivisible
  - ejemplos: while,if,==,>=,ancho,...
- **La secuencia de caracteres correspondiente se llama “lexema” (componente léxico)**
- **1 token <> 1 lexema**
- **Existen Generadores automáticos de analizadores léxicos como LEX**

El analizador léxico

- **Además, suele realizar otras tareas:**
  - procesar directivas al compilador (opciones)
  - introducir información preliminar en la tabla de símbolos
  - eliminar separadores innecesarios
  - sustituir macros
  - listar el fuente
- **Normalmente, los tokens se describen mediante expresiones regulares**

- Análisis Léxico (Escaneo)

- El <nombre-token> es un símbolo abstracto que se utiliza durante el análisis sintáctico, y el <valor-atributo> apunta a una entrada en la tabla se símbolos para este token. La información de dicha tabla se necesita para el análisis semántico y la generación de código.

Tabla de símbolos	
1	Token1
2	Token2
3	token3

El analizador léxico

- **Ejemplo**

```
posición = inicial + velocidad * 60
```

- **El scanner deberá reconocer sucesivamente,**

token	lexema
IDENTIFICADOR	posición
'='	=
IDENTIFICADOR	inicial
'+'	+
IDENTIFICADOR	velocidad
'*'	*
CONSTANTE	60

El analizador sintáctico

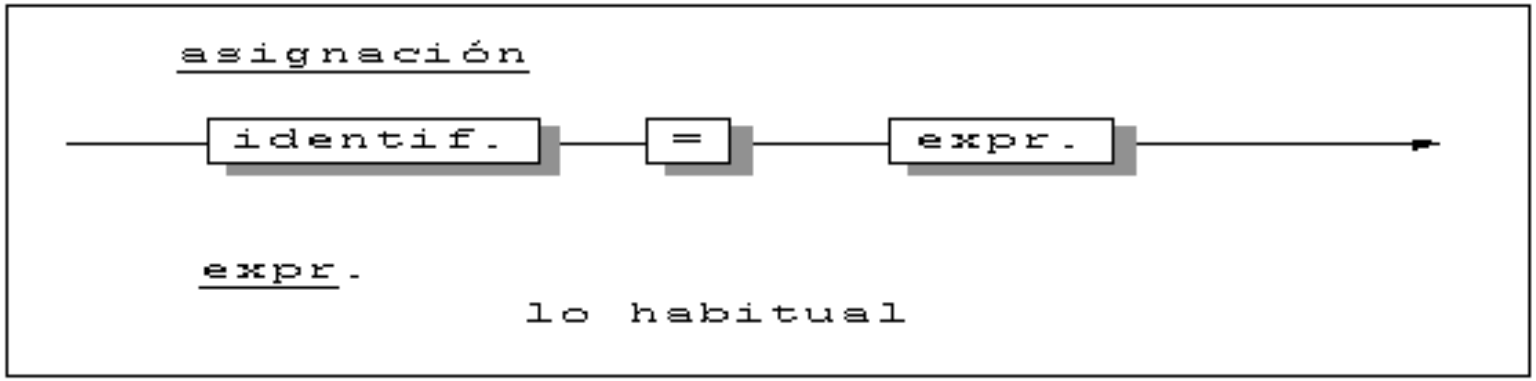
- **O “parser”**
- **Objetivo: agrupar los tokens suministrados por el scanner para reconocer “frases”**
- **¿Cómo lo hace?**
  - La sintaxis se suele especificar formalmente mediante una Gramática
  - El parser recibe tokens y los agrupa de acuerdo a especificadas por la GLC

- **El parser detecta errores sintácticos**

  Y si es bueno, puede además realizar algunas correcciones

El analizador sintáctico

- **Ejemplo:**
  - supongamos sintaxis asignación como:



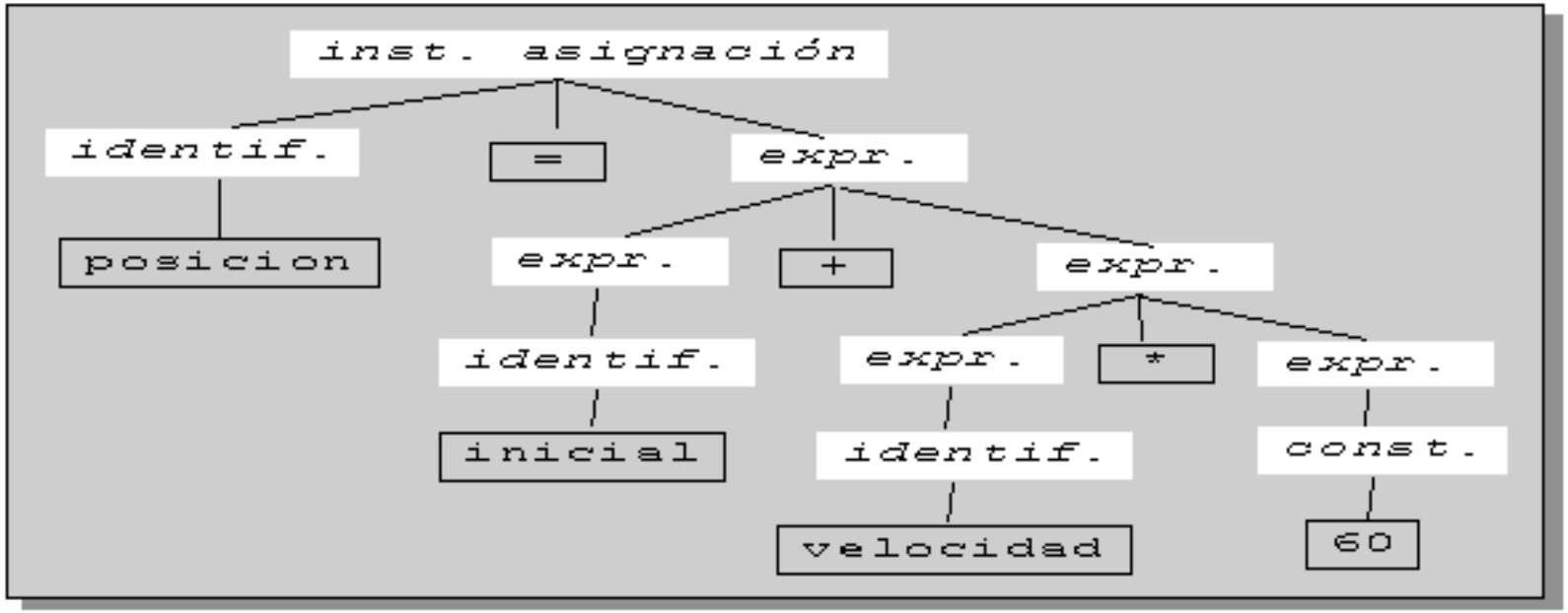
- y debemos analizar

```
posicion = inicial + velocidad * 60
```

El analizador sintáctico

- **El árbol sintáctico correspondiente es**

```
posicion = inicial + velocidad * 60
```



El analizador semántico

- **Realiza dos funciones:**
  - Análisis de la semántica
    - ¿Es cada construcción legal y “con sentido”?
      - variables en una expresión definidas
      - del tipo adecuado
      - alcance de los objetos
  - Generación del código intermedio
    - Generalmente se lleva a cabo mediante gramáticas de atributos
      - la GLC se completa con atributos necesarios
        - tipo
        - valor
        - acciones a ejecutar cuando se detecta una construcción legal

El analizador semántico

- Utiliza el árbol sintáctico y la información en la tabla de símbolos para comprobar la consistencia semántica del programa fuente con la definición del lenguaje. Realiza comprobación de tipos, donde verifica que cada operador tenga operandos que coincidan.

resul = x+ -y;

y=-2;
resul = x + y;

Generación de Código Intermedio

La generación de código intermedio transforma un árbol de análisis sintáctico (semántico) en una representación en un lenguaje intermedio, que suele ser código suficientemente sencillo para poder luego generar código máquina.

```
While (A>B) AND (A<=2*B-5)
 Do A := A + B

L1: IF A>B GOTO L2
      GOTO L3
L2: T1:= 2*B
      T2:= T1-5
       IF A<=T2 GOTO L4
       GOTO L3
L4: A:= A+B
      GOTO L1
L3: ....
```

OPTIMIZACION DE CODIGO

- **El código intermedio generado es analizado y transformado en uno equivalente optimizado**
- **Es una tarea muy costosa**
  - De hecho, generalmente se puede invocar al compilador activando/desactivando esta opción

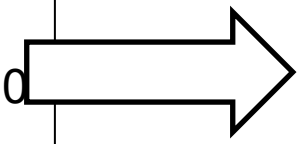
GENERACION DE CODIGO

- Toma código intermedio y genera código objeto para la máquina considerada
- Es la parte más próxima a la arquitectura de la máquina
- Dada la complejidad, si no se va a realizar optimización, se asocia la generación de código a las rutinas semánticas

GENERACION DE CODIGO

- Se recibe una representación intermedia del programa fuente y la asigna a la lengua destino. Después, las instrucciones intermedias se traducen en secuencias de instrucciones de maquina.

```
LDF     R2,     id3
MULF    R2,     R2,     #10
LDF     R1,     id2
ADDF    R1,     R1,     R2
STF     id1,    R1
```



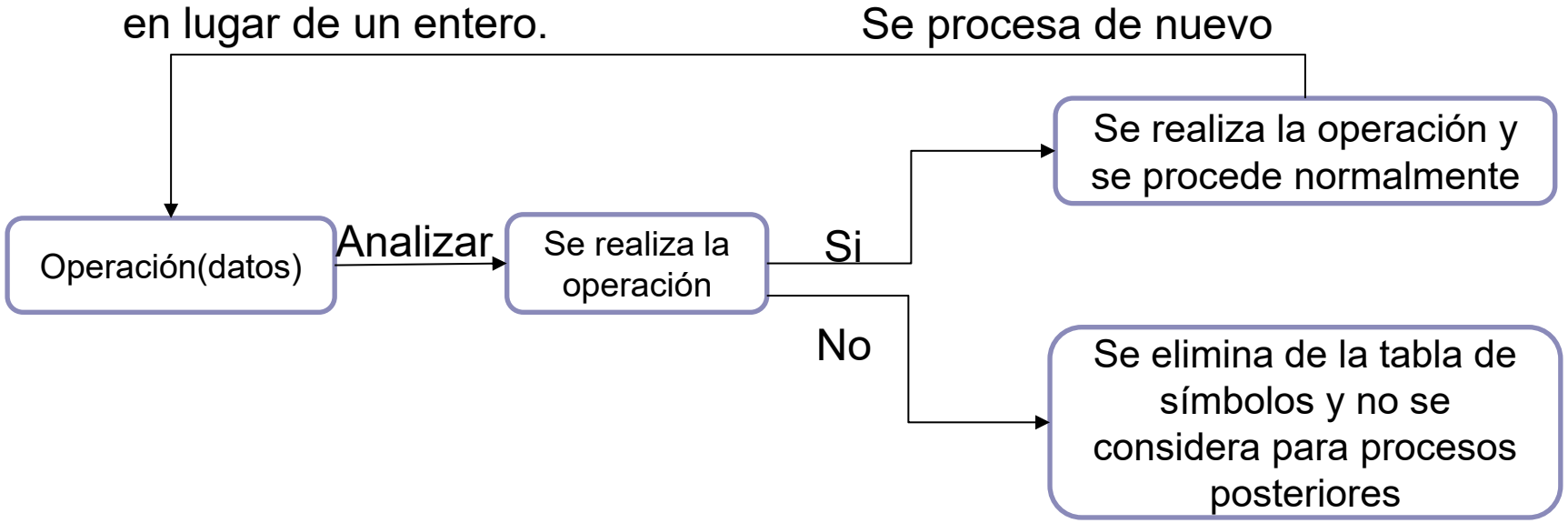
```
10011  11011  01101
01101
11001  01010  01101
11100
01101  11110  00011
00100
10110  00111  01011
01110
10011  01101  10110
01101
01101  11010  10001
01101
```

La tabla de símbolos

- Mecanismo para almacenar/acceder la información de los identificadores
- Las informaciones asociadas a un identificador se denominan “atributos”
- Cada vez que se usa un identificador, la tabla de símbolos proporciona la información necesaria
- Se usa tanto en la parte de análisis como en la de síntesis

La tabla de símbolos

Es un Array ampliable de registros que pueden ser indexados por una cadena en lugar de un entero.



Manejo de errores

Es una de las misiones mas importantes de un compilador, aunque al mismo tiempo, es lo que mas dificulta su realización debido principalmente a dos motivos:

1) A veces unos errores ocultan otros
2) A veces un error provoca una avalancha de muchos errores que se solucionan con el primero

Criterios en el manejo de errores

a) Pararse al detectar el primer error
b) Detectar todos los errores de un pasada

¿Dudas?